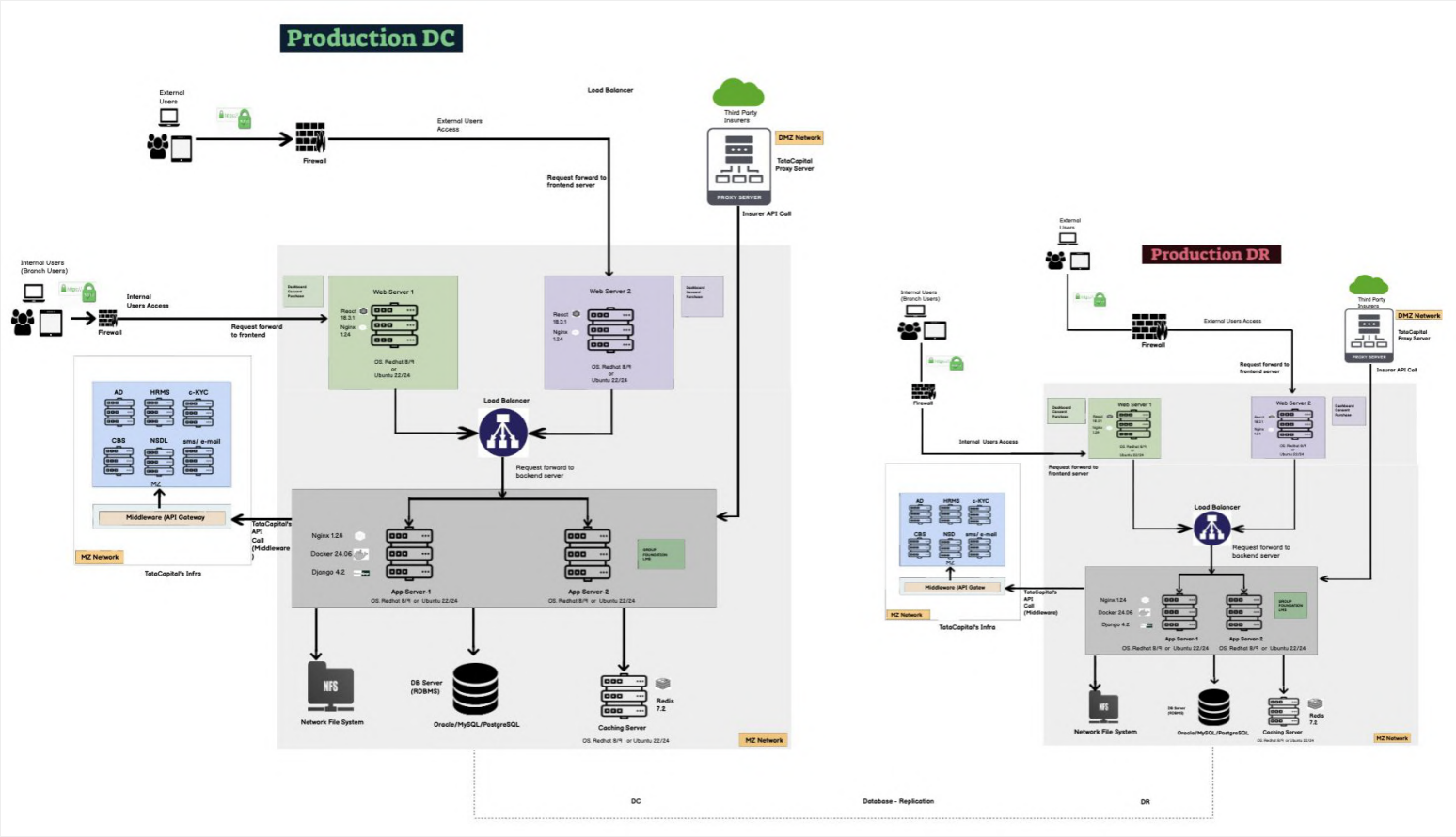
Technology: Solution Architecture & Deployment View

Technology: Deployment View (Production DC + DR, On-premise)



Integration Architecture

Supported Protocols:
The application primarily uses REST APIs for integration with both internal and external systems. REST provides a lightweight, scalable way to communicate over HTTP/HTTPS, allowing seamless data exchange in formats like JSON. This protocol is well-suited for web services due to its simplicity and flexibility, enabling easy integration across different platforms.

Asynchronous Processing:
The system supports asynchronous processing via Celery, which allows for efficient handling of background tasks without blocking the main application. By offloading long-running tasks to Celery, the application remains responsive and can scale to handle a high volume of concurrent operations. Celery integrates well with message brokers like Redis or RabbitMQ, ensuring reliable task distribution. This approach optimizes system performance by processing tasks in parallel without affecting the user experience.

Service Security:
The application ensures comprehensive service security through regular code reviews, robust access control, and timely patching to address vulnerabilities. Access to critical systems is tightly controlled, using role-based access or other mechanisms to limit permissions. The system is continuously assessed for security threats, and any vulnerabilities found are addressed promptly with patches. This multi-layered security approach safeguards the application from unauthorized access and ensures data integrity.

External System Integration:
External system integration is managed via a Proxy Server (Squid), which acts as a secure intermediary between internal systems and external services. The proxy server helps manage traffic flow, ensuring that communication between systems is properly secured and monitored. It provides features like content filtering, access logging, and caching, optimizing performance and security.

Technology

Service Exposure: The solution allows functionality to be exposed as real-time services via RESTful APIs. These APIs enable seamless communication between systems, ensuring efficient data exchange and real-time updates for both internal and external integrations

Integration Capabilities: Integration with other systems is supported using APIs and direct connectors. This enables interoperability with third-party services, such as payment gateways, SMS providers, and core banking systems, ensuring a cohesive ecosystem for end-to-end operations.

API Structures: The organization primarily uses RESTful APIs for communication. These APIs are designed with scalability and simplicity in mind, supporting standard HTTP methods (GET, POST, PUT, DELETE) to ensure compatibility across platforms.

Messaging Format: The system supports synchronous (real-time) and asynchronous (event-driven) messaging formats. API message formats include application/json for structured data exchange and urlencoded for lightweight payloads, ensuring flexibility based on use cases.

Open Web Standards: Exposed APIs comply with Open Web Standards, ensuring accessibility, security, and ease of integration. These standards allow third-party developers and systems to interact with APIs using common protocols like HTTP and HTTPS.

UI Design: The user interface is designed with a user-centric approach, offering intuitive navigation and workflows. This design minimizes learning curves and facilitates faster data entry, boosting overall operational efficiency.

Technology

Microservices Design Principles (RESTful API)

- Stateless communication using HTTP methods (GET, POST, PUT, DELETE).
- Clear, resource-based URLs and lightweight data formats like JSON/XML.
- Client-server separation: UI handled by the client, logic by the server.
- Built-in caching and error handling with HTTP status codes.

Alerting Mechanisms:

- Real-Time Alerts: Configured in tools like Zabbix, Nagios, AWS CloudWatch, Grafana, and Percona.
- Notification Channels: Alerts via email, SMS, or Slack ensure immediate team visibility.

Browser Compatibility

The solution is thoroughly designed and tested to ensure compatibility with the latest versions of all major web browsers. These include Google Chrome, Safari, Microsoft Edge, and Mozilla Firefox. By adhering to modern web standards such as HTML5 and CSS3, the solution ensures a seamless and consistent user experience across all supported platforms. Additionally, robust testing is conducted to address any browser-specific nuances, guaranteeing that the functionality and performance remain optimal, regardless of the user's preferred browser.

Web Features and Plugins

The solution supports modern web features like responsive design for adaptability across devices and Progressive Web Apps (PWA) for offline functionality and push notifications. Additionally, it integrates seamlessly with third-party plugins such as analytics tools, payment gateways, and chatbots, enabling businesses to enhance and customize their capabilities as needed.

Event Generation

The system efficiently handles event generation and complex event processing (CEP) to enable real-time automation and decision-making. It supports real-time event triggers for workflows like notifications and dashboard updates, while CEP analyzes event patterns for actionable insights. Built on scalable frameworks such as Apache Kafka and Apache Flink, the system ensures reliability with retries, deduplication, and prioritization, making it ideal for high-throughput environments.

Data Design, Storage and Processing

Data Encryption: All Personally Identifiable Information (PII) is stored in encrypted formats using industry-standard encryption protocols (e.g., AES-256). For data in transit, secure protocols such as TLS 1.2/1.3, SSL digital certificate and payload encryption are used for encrypting data during transmission between services.

Data Deduplication: The system ensures a single source of truth for member data by leveraging the bank's APIs to consolidate and maintain accurate data across insurance objects, segments, and channels. Duplicate records are identified and merged, ensuring clean and consistent data.

Data Integrity: Integrity is maintained through database constraints and validation at every stage of processing.

Extendable Data Model: The data models are designed to be future-proof and can be extended to accommodate evolving business needs without disrupting existing functionality.

Error Logging and Alerts: The system includes robust error logging and configurable alerting mechanisms. Critical errors are captured and logged with full audit trails, providing detailed information about the nature and context of the error. Additionally, alerts are configured to notify relevant stakeholders via email or SMS, ensuring that any system failures or performance issues are addressed promptly. This proactive monitoring helps prevent major disruptions and maintains system stability..

Batch Error Handling: Batch errors are effectively handled to ensure that data consistency is maintained throughout the process. When a failure occurs, the system identifies the problematic transactions and automatically revisits or reprocesses them to correct the issues. This ensures that no data is left in an inconsistent state and that the overall batch process completes successfully, maintaining data integrity across the system.

Scalable Architecture using Microservices

This solution embraces a microservices architecture, where distinct services handle specific business functions like lead generation, payments, and reporting.

This approach offers several key advantages:

Independent Services: Each module operates as an independent unit. This modularity enhances flexibility, simplifies development and maintenance, and allows for targeted updates without affecting other services.

Scalability on Demand: Microservices can be scaled individually based on demand. By leveraging containerization technologies like Docker and orchestration platforms like Kubernetes or OpenShift, resources can be allocated dynamically to meet fluctuating needs.

Seamless Integration: Integration with external systems (e.g., Core Systems, Active Directory, KYC providers) and partner platforms is achieved through well-defined APIs or message queues like Kafka or RabbitMQ. This enables interoperability and streamlines data exchange.

Flexible Communication: Services communicate using both synchronous (REST/gRPC APIs) and asynchronous (event-driven queues) mechanisms. This allows for efficient handling of both real-time requests and background processes.

Decentralized Data Management: Each microservice can have its own dedicated database, optimized for its specific needs. This approach improves performance, ensures data ownership, and reduces dependencies between services.

This microservices architecture promotes agility, scalability, and resilience, enabling the system to adapt to changing business requirements and handle increasing workloads effectively.

Data Design, Storage and Processing

Real-Time Process Management

Real-time processes support the ability to pause, restart, and cancel without affecting data integrity. This functionality is managed through a flexible product configurator that enables seamless control over ongoing processes. By enabling or disabling specific products, administrators can pause or restart processes as needed, ensuring that the system remains stable while allowing for the necessary adjustments without data loss or corruption.

Batch Processing

The system supports batch processing, which is efficiently managed using CRON jobs for periodic tasks such as policy status updates. These jobs run on scheduled intervals, ensuring that data is processed and updated regularly without human intervention. The use of CRON ensures that the process is automated, reducing the risk of errors while maintaining the accuracy of data during each batch operation.

Batch Process Management

Batch processes are designed to support pause, restart, and cancel modes, ensuring that data integrity is maintained throughout the batch operation. Critical tasks related to data synchronization can be managed effectively in batches, allowing flexibility for users to control the process if needed. These capabilities ensure that ongoing tasks are not disrupted, and any issues that arise can be resolved without compromising the consistency or accuracy of the data.

Technology

Stack:

The solution is built using industry-standard technologies:

Frontend: ReactJS for web interfaces, Flutter for cross-platform mobile apps, and native development for Android/iOS.

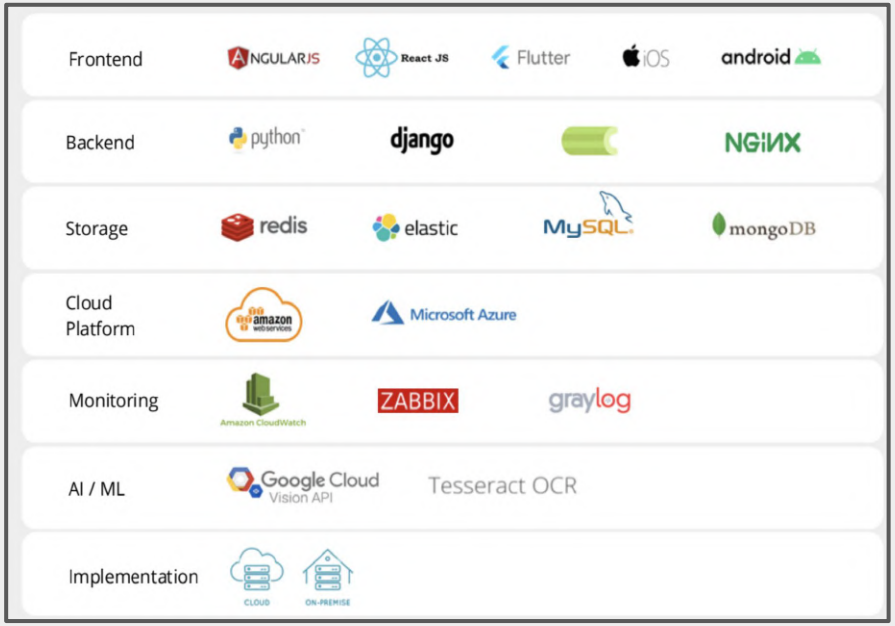
Backend: Python/Django framework for scalability and robustness.

Caching/Queueing : Redis for efficient caching and task queuing.

Database: MySQL or similar SQL database for structured and reliable data storage.

Monitoring: Tools include Zabbix for infrastructure, Graylog for logs, CloudWatch for cloud metrics, Percona for database performance, and Uptime for availability.

Infrastructure: Both on-premise and private cloud are supported.



Solution Design

Deployment Models: This architecture utilizes a hybrid cloud active-active model, distributing workloads across two AWS regions in India: Mumbai (primary) and Hyderabad (disaster recovery). This ensures high availability and resilience. Protected VPCs (Virtual Private Clouds) are employed to create secure and isolated environments for resources.

PII Data Storage & Access: Personally Identifiable Information (PII) is stored securely in Oracle 19c databases located within private subnets, restricting direct access from the public internet. Data is protected with encryption managed by AWS Certificate Manager, and access is tightly controlled using AWS Identity and Access Management (IAM). Real-time monitoring of security threats and suspicious activities is implemented through AWS GuardDuty and CloudWatch.

Multi-Entity Hosting: The system supports multi-tenancy, meaning it can host multiple separate entities or clients. This is achieved by providing each tenant with dedicated web servers, application servers, and databases. Load balancers (both public-facing and internal) are used to distribute traffic efficiently and ensure scalability as demand grows.

Integration Capabilities: Internal Application Load Balancers (ALBs) facilitate seamless communication between different system components within the private subnets. External integrations are secured through public ALBs, AWS Web Application Firewall (WAF), and services like Amazon Simple Notification Service (SNS) and Amazon Simple Storage Service (S3).

Project Execution Plan: Automation is a key principle, with Infrastructure as Code (IaC) tools like Terraform or CloudFormation managing deployments and Amazon Elastic Container Registry (ECR) handling containerized applications. Testing is integrated throughout the process, with automated pipelines for integration, regression, and performance testing. Deployments aim for zero downtime, leveraging load balancers and phased rollouts to minimize disruption to users. This comprehensive design prioritizes security, scalability, and operational efficiency while providing robust support for multi-tenancy and integrations.

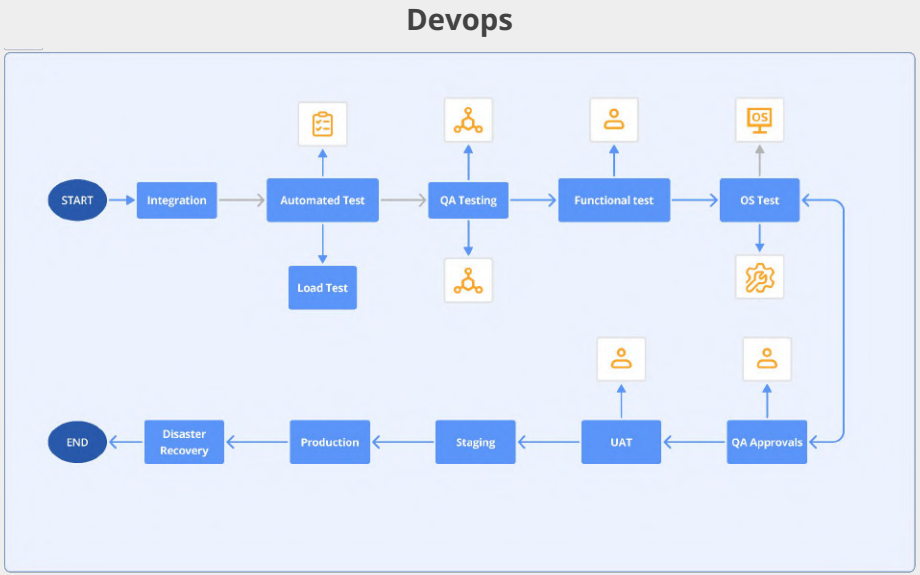
Technology

DevOps Implementation Overview

1. Framework
Source Code Management: Git for version control. Build Automation: Jenkins for CI/CD pipelines. Monitoring Tools include Zabbix for infrastructure, Graylog for logs, CloudWatch for cloud metrics, Percona for database performance, and Uptime for availability.

2. Deployment Workflow
Developers commit code to a Git repository (platforms like GitHub or GitLab).
Jenkins is triggered via webhooks for CI/CD pipelines. Code is built, unit-tested, and packaged into Docker containers.

3. Monitoring and Change Management
Dashboards track system performance, and alerts notify of critical issues. Changes are approved via Jira/ServiceNow, with Blue-Green deployment minimizing downtime.



Technology

CI/CD Pipeline: Detailed Architecture

Our CI/CD pipeline ensures seamless automation from code development to deployment across all environments (Dev, UAT, Preprod, and Production). It incorporates the following stages:

Code Commit: Developers commit code to a centralized version control system (Git). Each commit triggers the CI/CD pipeline automatically, ensuring continuous integration and delivery.

Build Stage: Using Jenkins as the orchestration tool, the pipeline compiles the code, resolves dependencies, and creates build artifacts. This step ensures the build is stable before proceeding further.

Test Stage: Automated unit, integration, and functional tests are executed in this stage. It validates the code's functionality and identifies defects early in the development lifecycle.

Stage Environment: Once the build passes all test cases, it is deployed to the staging environment for further validations, such as performance testing and user acceptance testing (UAT). This step ensures readiness for production.

Deploy Stage: The tested and validated build is automatically promoted to higher environments (Production) without manual intervention. AWS CodePipeline handles the deployment, ensuring scalability, rollback capabilities, and high availability.

